VLSI Design Automation (EECE 5186C/6086C)

Ranga Vemuri

Home Work – 2

In this homework, you will implement a router. This is a group homework. Two students per group as in HW-1.

You will be given netlists in the following Python dictionary format:

```
data = {
      'grid_size': int,
      'nets': {
          'NET_i': {
              'pins': [(x1,y1), (x2,y2)],
              'type': 'LOCAL|MEDIUM|LONG',
              'length': int }}}
```

Routing grid is assumed to be a square. grid_size provides the width and height of the routing grid. All nets are two terminal nets. All terminals are assumed to be in M1. For each net, coordinates of its two terminals are provided. Coordinate positions are numbered from 0 to grid_size-1. Net length is the rectilinear distance between two terminals: |x1-x2| + |y1-y2|. Length is provided only for cross checking if needed. Net type should be ignored. The router is free to classify the nets as appropriate for the routing methodology.

See the following example in the file netlist_100_3.py:

```
data = {   'grid_size': 100,
           'nets': {   'NET_0': {   'length': 3,
                                    'pins': [(99, 56), (99, 59)],
                                    'type': 'LOCAL'},
                       'NET_1': {   'length': 48,
                                    'pins': [(5, 72), (44, 63)],
                                    'type': 'MEDIUM'},
                       'NET_2': {   'length': 52,
                                    'pins': [(43, 74), (5, 60)],
                                    'type': 'LONG'}
                   }
       }
```

In this example, grid size is 100x100. There are three nets whose pin positions are provided. Length and type can be ignored.

Given a netlist, the router's task is to route all the nets in metal layers M2-M9. Even numbered layers are used exclusively for vertical wire segments and odd numbered layers are used exclusively for horizontal wire segments.

Each via costs 2 units. Cost of a wire segment is its length (that is, each grid cell costs 1 unit). Goal of the router is to route all the nets and minimize the total path cost of all the nets combined.

Routing paths should be reported in an output file in the following format:

```
data = {
    "meta": {
        "grid_size": int,
        "layer_directions": {"M2":"V", "M3":"H", ...},
        "total_cost": int
    },
    "nets": {
        "NET_0": {
            "pins": [(x1,y1), (x2,y2)],
            "segments": [...],
            "cost": int
        }
    }
}
```

Meta data includes 'grid_size' (same as the size in the input file), directions for each layer as specified above, and 'total_cost' which should be the sum the path costs of all the nets. For each net, the pin positions should be the same as the ones in the input file. 'segments' is the series of segments (including horizontal/vertical wires and vias) in the path from one pin to the other. 'cost' of the net is the path cost including the via costs plus the wire segment costs.

File netlist_100_3_reference_route.py is an example post-route file corresponding to the input file netlist_100_3.py. A portion of this post-route file is shown below:

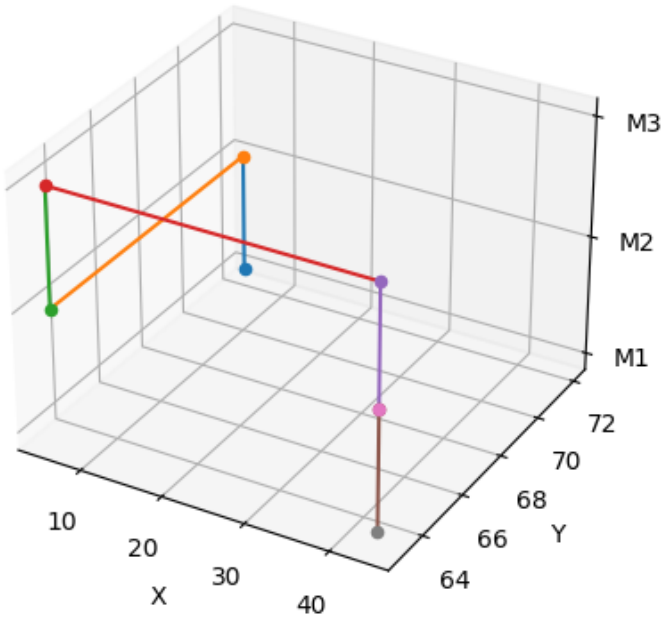
```
data = {   'meta': {   'grid_size': 100,
                'layer_directions': {   'M2': 'V',
                                        'M3': 'H',
                                        'M4': 'V',
                                        'M5': 'H',
                                        'M6': 'V',
                                        'M7': 'H',
                                        'M8': 'V',
                                        'M9': 'H'},
                 'total_cost': 123},

    'nets': {   'NET_0': {…},
                'NET_1': {   'cost': 56,

                             'pins': [(5, 72), (44, 63)],

                             'segments': [   {   'end': (5, 72, 'M2'),
                                                 'start': (5, 72, 'M1')},
                                             {   'end': (5, 63, 'M2'),
                                                 'start': (5, 72, 'M2')},
                                             {   'end': (5, 63, 'M3'),
                                                 'start': (5, 63, 'M2')},
                                             {   'end': (44, 63, 'M3'),
                                                 'start': (5, 63, 'M3')},
                                             {   'end': (44, 63, 'M2'),
                                                 'start': (44, 63, 'M3')},
                                             {   'end': (44, 63, 'M1'),
                                                 'start': (44, 63, 'M2')}]},
                'NET_2': {…}
            }
        }
```

In the 'meta' data portion, grid_size should be copied from the input file and layer_directions should exactly be as shown above. Layers unused in a specific netlist may be omitted. In the 'nets' portion, every net in the input file should be present. 'pins' for each net should list the coordinates as in the input file. 'segments' shows the list of segments in the path. Each segment has a 'start' and 'end.' Both specified specified as a coordinate position and a layer id.

Each segment between (xa, ya, Mi) and (xb, yb, Mj) should be of one of the following forms:

- Via: xa = ya, xb = yb, |i-j| = 1
- Vertical wire segment: xa = xb, i = j = 2, 4, 6, or 8.
- Horizontal wire segment: ya = yb, i = j = 3, 5, 7, or 9

The segments are listed in the order in which they are connected from the first pin to the second pin. Since pins are in M1, the first segment should be a via from M1 to M2. Similarly, the last segment should always be a via from M2 to M1. Net_1 route is illustrated in the figure.



'cost' of a net is its path cost, including the cost of all vias and all wire segments. Each via cost is 2. Wire segment cost is its length.

You will receive two sets of netlists:

- Rtest: For some of these netlists, we are aware of feasible path costs (optimality unknown) and for others we can provide potential target path costs (feasibility and optimality unknown).

Sl.	Netlist	Grid Size	Nets	Target Path Cost	Is the Target Feasible?
1	Rtest_100_100.py	100	100	21,000	Yes
2	Rtest_100_400.py	100	400	4,506	Yes
3	Rtest_100_1000.py	100	1000	10,513	Yes
4	Rtest_100_250.py	100	250	6,975	Unknown
5	Rtest_100_500.py	100	500	14,053	Unknown
6	Rtest_500_5000.py	500	5,000	61,162	Yes
7	Rtest_500_6000.py	500	6,000	247,523	Unknown
8	Rtest_1000_20000.py	1,000	20,000	245,144	Yes
9	Rtest_1000_25000.py	1,000	25,000	1,438,995	Unknown
10	Rtest_1500_45000.py	1,500	45,000	551,397	Yes
11	Rtest_1500_50000.py	1,500	50,000	3,728,878	Unknown

You can use Rtest netlists for testing and tuning your programs.

- Reval: For these netlists we have no information about the potential path costs.

Sl.	Netlist	Grid Size	Nets
1	Reval_100_400.py	100	400
2	Reval_500_5000.py	500	5,000
3	Reval_1000_20000.py	1000	20,000
4	Reval_1000_30000.py	1000	30,000
5	Reval_1000_40000.py	1000	40,000
6	Reval_1000_60000.py	1000	60,000
7	Reval_1000_100000.py	1000	100,000
8	Reval_1500_40000.py	1500	40,000

Your programs are expected to be in Python or C++. You can develop your programs on any machine but the final results should be gathered on the VLSI Lab machines. Python3 and VSCode are available on these machines. C++ compilers are also available.

You need to evaluate your programs on both sets of netlists and report results. You should not allow more than three hours of CPU time for any netlist on the VLSI Lab machines.

Your submission must be a single .zip file and contain the items described below:

1. Your source code, properly commented and indented for easy readability and submitted as a single file. Name this <LastName1>_<LastName2>_Router.py. Include execution instructions at the top.

2. Name the result files as <OriginalNetlistName>_route.py. Put these files in two separate folders and name the folders <LastName1>_<LastName2>_<OriginalFolderName>. Result files should be pretty printed for easy readability similar to the reference route file given.

3. For random number generation, use a seed and hardcode the seed value. Use the same seed for all runs. Reproducibility of results is essential.

4. A report as a pdf file. It should contain:

    1. Student names and M numbers.
    2. Description of the overall methodology, algorithm(s), key implementation decisions, tuning procedure used, and the final parameters.
    3. Result tables indicating the quality of the result (number of nets routed, highest metal layer used, total path cost), CPU and memory requirements for each netlist.
    4. Any other information and experimental data you feel necessary to judge the quality of your algorithm, your software, and your effort.
    5. A retrospective describing what you think is the cause of the good/poor performance of your tool and how might you be able to improve the performance (both execution speed and quality of result) of your tool if you had more time?

Note on use of AI Tools for this homework only: You are allowed to use AI tools for coding (but not report writing) provided you describe which tool was used and how. You are required to share your chat interactions if asked. All other academic misconduct rules are in effect.

You may be asked to give a demo/presentation on your projects and required to answer any questions.